

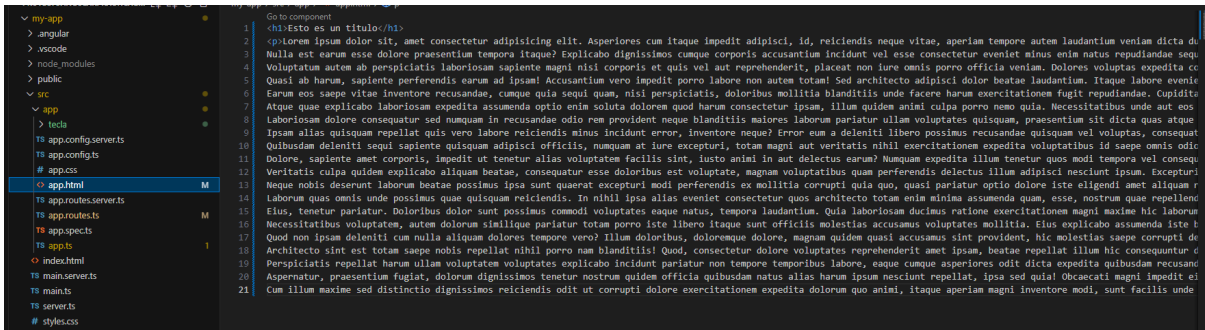
Angular desde 0

Generacion de Proyecto

```
ng new nombre_proyecto
```

Esto genera la **estructura del proyecto**. Se trabaja sobre **app.html (app.component.html** en versiones antiguas).

En la siguiente imagen se ve el archivo **app.html** que agregamos un título y un lorem, en la foto que le sigue vemos en la página web ya renderizada.



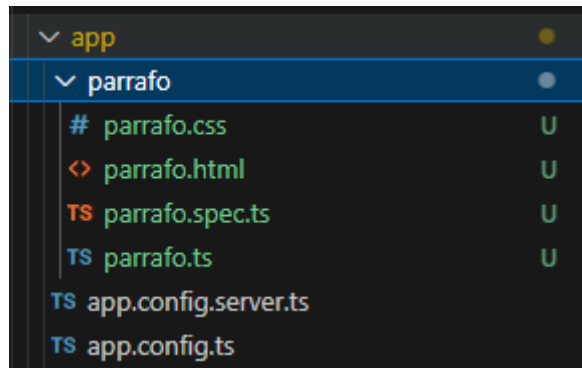
Esto es un título

Lorem ipsum dolor sit, amet consectetur adipiscing elit. Asperiores cum itaque impedit adipisci, id, reiciendis neque vitae, aperiam tempore autem laudantium veniam dicta ducimus maiores eligendi! Beatae amet quae facilis? Nulla est eorum esse dolore praesentium tempora itaque? Explicabo dignissimos cumque corporis accusantium incident vel esse consectetur eveniet minus enim natus repudiandae sequi, Quasi ab harum, sapiente perferendis eorum ad ipsam! Accusantium vero impedit porro labore non autem totam! Sed architecto adipisci dolor beatae laudantium. Itaque labore eveniet Earum eos saepe vitae Inventore recusandae, cumque quia sequi quam, nisi perspicatis, doloribus mollitia blanditiis unde facere harum exercitationem fugit repudiandae. Cupiditate Atque quae explicabo laboriosam expedita assumenda optio enim soluta dolorem quod harum consectetur ipsum, illum quidem animi culpa porro nemo quia. Necessitatibus unde aut eos laboriosam dolore consequatur sed numquam in recusandae odio rem provident neque blanditiis maiores laborum pariatur ullam voluptates quisquam, praesentium sit dicta quas atque ipsam alias quisquam repellat quis vero labore reiciendis minus incident error, Inventore neque? Error eum a deleniti libero possimus recusandae quisquam vel voluptas, consequat Quibusdam deleniti sequi sapiente quisquam adipisci officiis, numquam at iure excepturi, totam magni aut veritatis nihil exercitationem expedita voluptatibus id saepe omnis odic Dolore, sapiente amet corporis, impedit ut tenetur alias voluptatem facilis sint, iusto animi in aut delectus eorum? Numquam expedita illum tenetur quos modi tempora vel consequ Veritatis culpa quidem explicabo aliquam beatae, consequatur esse doloribus est voluptate, magnam voluptatibus quam perferendis delectus illum adipisci nesciunt ipsum. Excepturi! Neque nobis deserunt laborum beatae possimus ipsa sunt quaeat excepturi modi perferendis ex mollitia corrupti quia quo, quasi pariatur optio dolore iste eligendi amet aliquam r laborum quas omnis unde possimus quae quisquam reiciendis. In nihil ipsa alias eveniet consectetur quos architecto totam enim minima assumenda quam, esse, nostrum quae repellendus Eius, tenetur pariatur. Doloribus dolor sunt possimus commodi voluptates eaque natus, tempora laudantium. Quia laboriosam ducimus ratione exercitationem magni maxime hic laborum Necessitatibus voluptates, autem dolorum similique pariatur totam porro iste libero itaque sunt officiis molestias accusamus voluptates mollitia. Eius explicabo assumenda iste b Quod non ipsam deleniti cum nulla aliquam dolores tempore vero? Illum doloribus, doloremque dolore, magnam quidem quasi accusamus sint provident, hic molestias saepe corrupti de Architecto sint est totam saepe nobis repellat nihil porro nam blanditiis! Quod, consectetur dolore voluptates reprehenderit amet ipsum, beatae repellat illum hic consequuntur d Perspicatis repellat harum ullam voluptatem voluptates explicabo incident pariatur non tempore temporibus labore, eaque cumque asperiores odit dicta expedita quibusdam recusandae Perspicatis repellat harum ullam voluptatem voluptates explicabo incident pariatur non tempore temporibus labore, eaque cumque asperiores odit dicta expedita quibusdam recusandae Aspernatur, praesentium fugiat, dolorum dignissimos tenetur nostrum quidem officia quibusdam natus alias harum ipsum nesciunt repellat, ipsa sed quia! Obcaecati magni impedit ei Cum illum maxime sed distinctio dignissimos reiciendis odit ut corrupti dolore exercitationem expedita dolorum quo animi, itaque aperiam magni inventore modi, sunt facilis unde itaque aperiam magni inventore modi, sunt facilis unde sint? Maiores aperiam iste ad saepe!

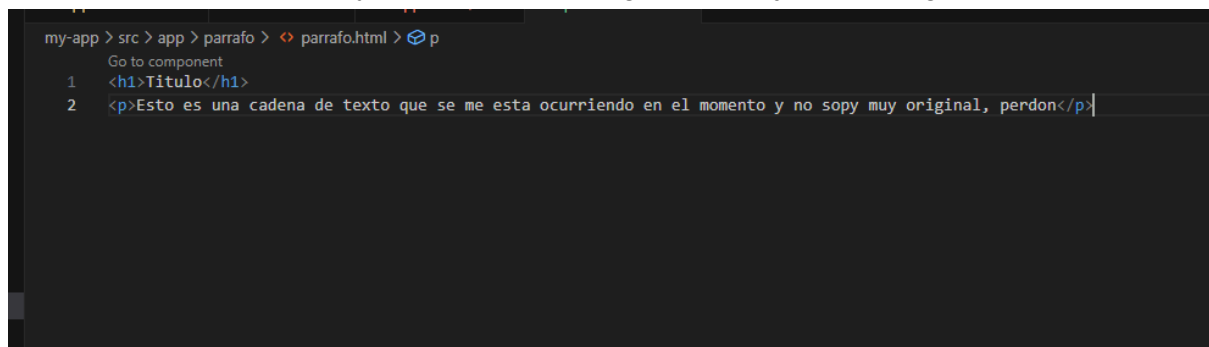
Crear módulo

Ahora, si tuviéramos que hacer muchas veces **título + lorem** sería muy extenso y complejo por lo que se empiezan a utilizar los **módulos** para reutilizar código. Para el siguiente ejemplo reemplazamos el lorem por una cadena más corta.

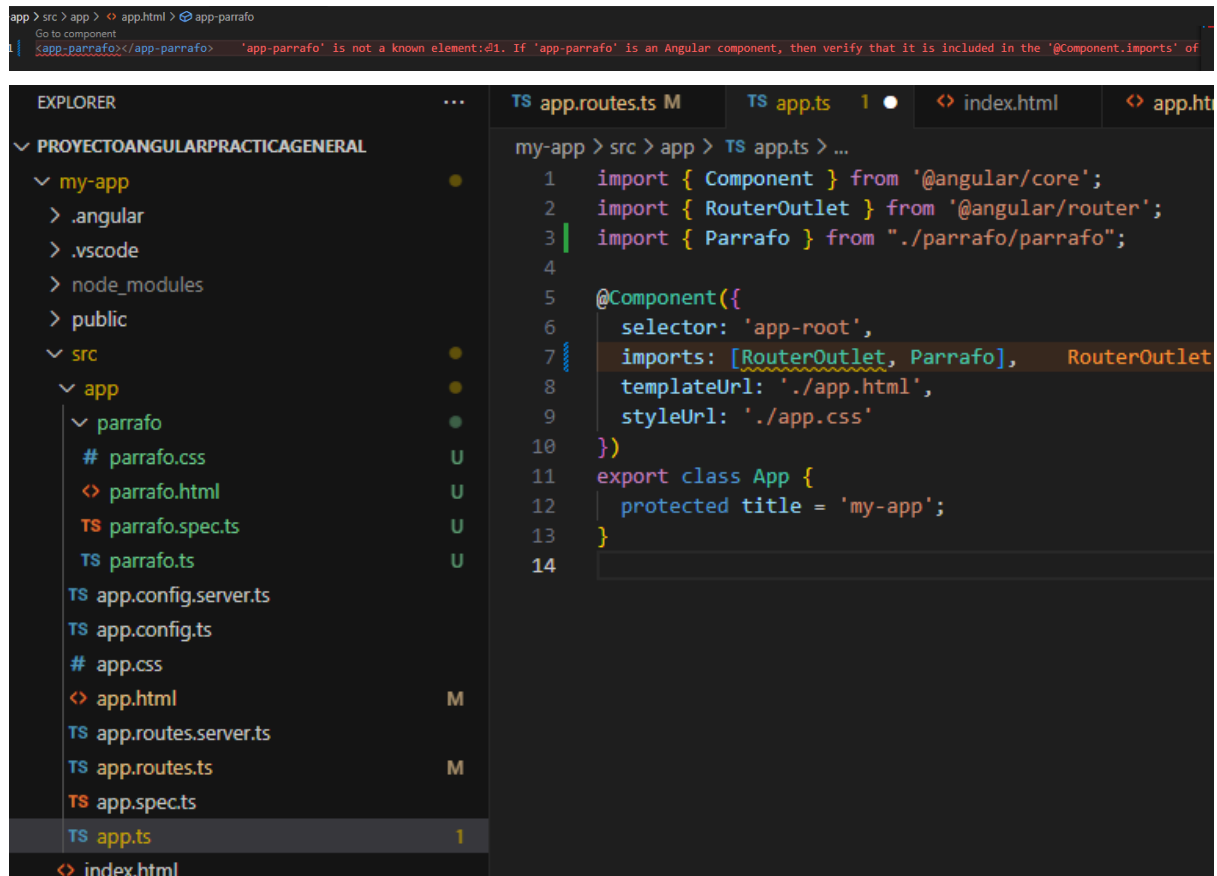
Mediante el CLI usamos **ng g c nombre_componente --standalone** lo que nos genera ya una carpeta en app con todo lo que necesita para existir. Nota: En este caso llamaremos párrafo al componente.



Aca modificaremos el html y aplicaremos css a gusto, por ejemplo el siguiente html



Pero esto no es suficiente para que desde **app.html** se renderise, como se ve en la siguiente imagen tira error. Lo que se debe hacer es **importarlo**, esto se hace en el archivo **app.ts**.



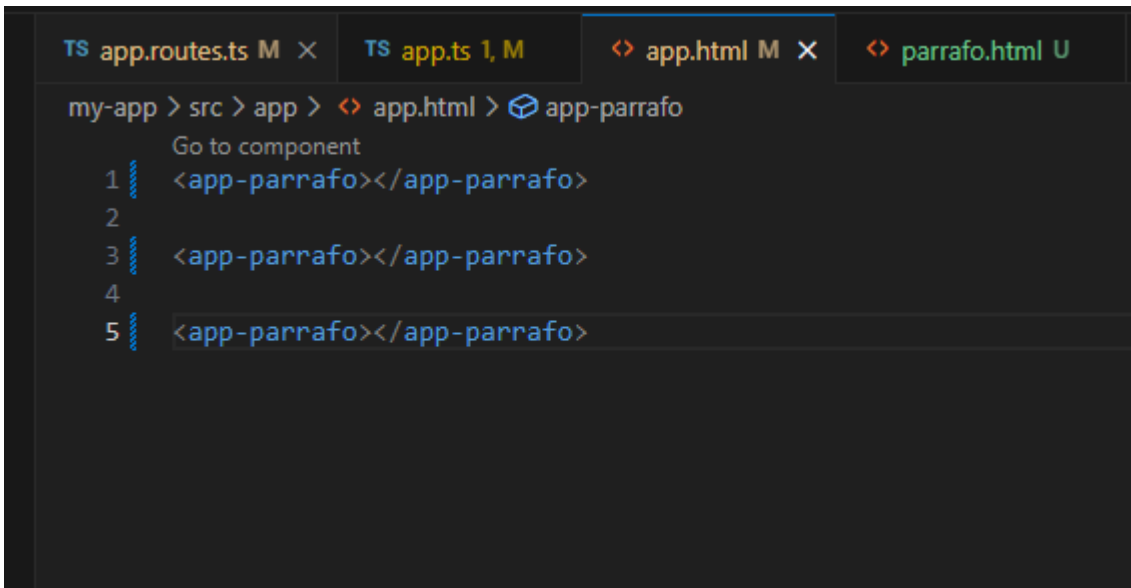
```
app > src > app > < app.html > app-parrafo
Go to component
<app-parrafo></app-parrafo> 'app-parrafo' is not a known element:❖1. If 'app-parrafo' is an Angular component, then verify that it is included in the '@Component.imports' of

EXPLORER
PROYECTOANGULARPRACTICAGENERAL
  my-app
    .angular
    .vscode
    node_modules
    public
    src
      app
        parrafo
          parrafo.css
          parrafo.html
          parrafo.spec.ts
          parrafo.ts
        app.config.server.ts
        app.config.ts
        app.css
        app.html
        app.routes.server.ts
        app.routes.ts
        app.spec.ts
        app.ts
      index.html

TS app.routes.ts M
TS app.ts 1 ●
index.html
app.html

my-app > src > app > TS app.ts > ...
1 import { Component } from '@angular/core';
2 import { RouterOutlet } from '@angular/router';
3 import { Parrafo } from './parrafo/parrafo';
4
5 @Component({
6   selector: 'app-root',
7   imports: [RouterOutlet, Parrafo], RouterOutlet
8   templateUrl: './app.html',
9   styleUrls: ['./app.css']
10 })
11 export class App {
12   protected title = 'my-app';
13 }
14
```

Hecho esto, ya se puede usar **<app-parrafo>** dentro de app.html y todo lo que se agregue a **parrafo.html** también se agregara cuando se llame a dicha etiqueta



Titulo

Esto es una cadena de texto que se me esta ocurriendo en el momento y no sopy muy original, perdon

Titulo

Esto es una cadena de texto que se me esta ocurriendo en el momento y no sopy muy original, perdon

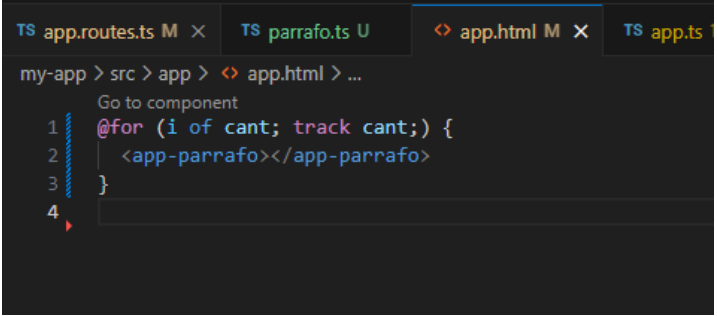
Titulo

Esto es una cadena de texto que se me esta ocurriendo en el momento y no sopy muy original, perdon

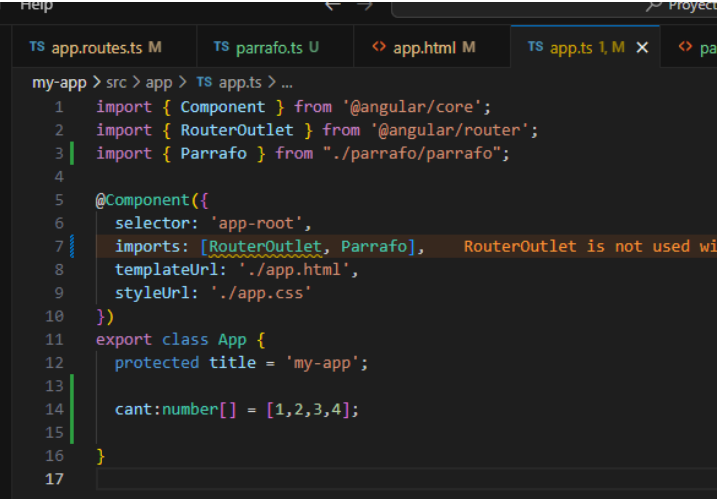
Ahora, estamos agregando manualmente muchas etiquetas, por lo que no se considera buena práctica, lo que se usaría en este caso es un for propio de angular (**ngFor**) para **mandar el componente varias veces**.

También aprovecharemos la **exportación en app.ts** para definir una **variable** que sea el **número de veces a repetir**. La única cosa es que necesita un **iterable**, osea, **necesita un array** para lo que generamos un array de n elementos, donde n es la cantidad de iteraciones que queremos.

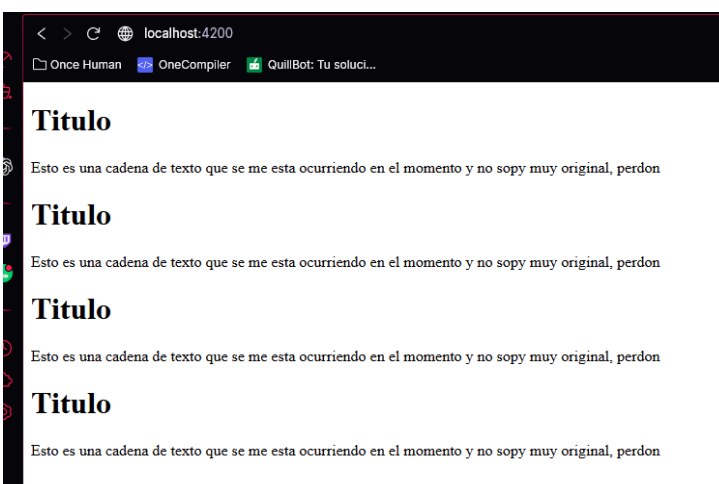
¿Dónde hacemos esto? Bueno, la **variable** que contenga el array la **definimos en app.ts**, **ya que el for esta en app.html**. Si fuera dentro del componente en si, se crearan dentro del .ts y .html del componente.



```
TS app.routes.ts M x TS parrafo.ts U <> app.html M x TS app.ts 1
my-app > src > app > <> app.html > ...
Go to component
1 @for (i of cant; track cant;) {
2   <app-parrafo></app-parrafo>
3 }
4
```



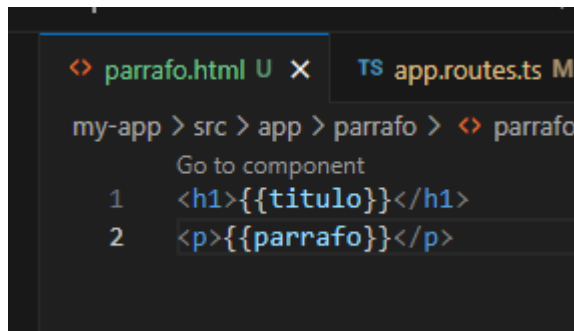
```
TS app.routes.ts M TS parrafo.ts U <> app.html M TS app.ts 1, M x <> pa
my-app > src > app > TS app.ts > ...
1 import { Component } from '@angular/core';
2 import { RouterOutlet } from '@angular/router';
3 import { Parrafo } from "../parrafo/parrafo";
4
5 @Component({
6   selector: 'app-root',
7   imports: [RouterOutlet, Parrafo], RouterOutlet is not used wi
8   templateUrl: './app.html',
9   styleUrls: ['./app.css']
10 })
11 export class App {
12   protected title = 'my-app';
13
14   cant:number[] = [1,2,3,4];
15
16 }
17
```



Iterar sobre elementos y hacer reemplazo

Llegado a este punto ya sabemos lo básico para generar elementos de forma repetida, pero **si queremos** que estos tengan **datos diferentes** tenemos que **meterlos en variables** y que estas se **rendericen en pantalla**. Vamos a modificar 4 archivos: [app.ts](#), app.html, [parrafo.ts](#) y parrafo.html.

En **parrafo.html** queremos usar la sintaxis de doble llave, que indica que en ese **lugar va a ir una variable**.



```
<h1>{{titulo}}</h1>
<p>{{parrafo}}</p>
```

En [parrafo.ts](#) vamos a tener que **indicarle que va a ser realmente esas variables** y de donde van a venir. **@Input()** vamos a decir que **toma valores de entrada** (No es que sea un input de html, sino que se refiere a entrada de datos), seguido del **nombre y el tipo (nombre:tipo)** y finalmente **inicializarlo vacío** para evitar errores.

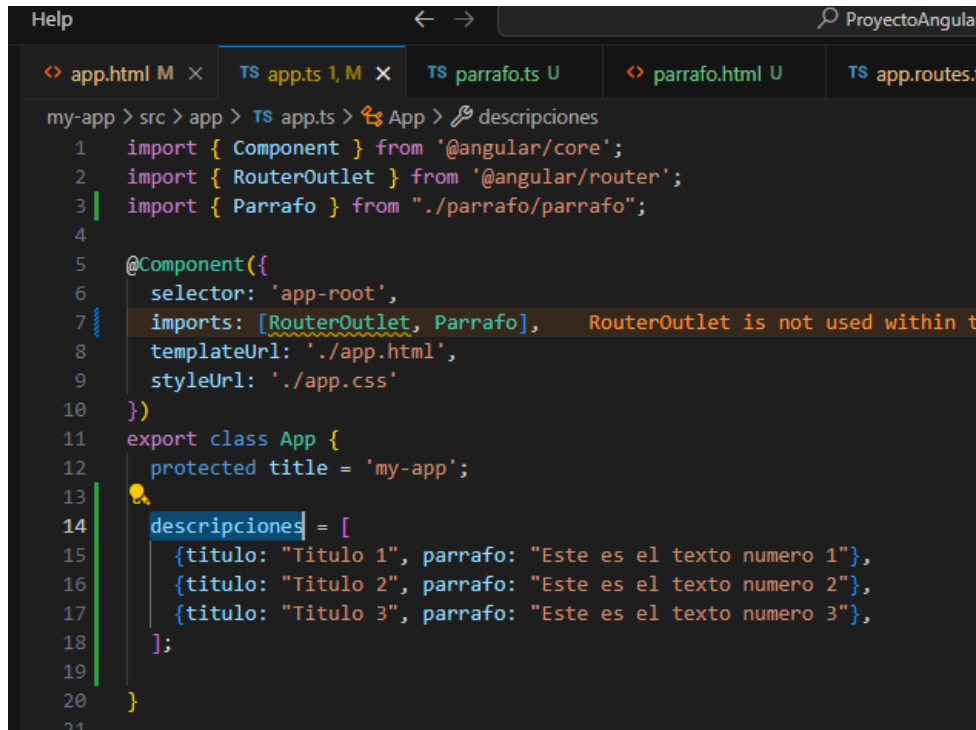


```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-parrafo',
  imports: [],
  templateUrl: './parrafo.html',
  styleUrls: ['./parrafo.css']
})
export class Parrafo {
  @Input() titulo: string = '';
  @Input() parrafo: string = '';
}
```

En [app.ts](#) vamos a modificar el array de números que tenían, para que contenga una **estructura de datos** más acorde a lo que buscamos para el componente.

Vamos a definir un array con la estructura de un **JSON** que tenga **título y párrafo**. Nota: La buena practica seria usar un objeto para poder tomar la variable cant, pero de momento lo dejamos así.

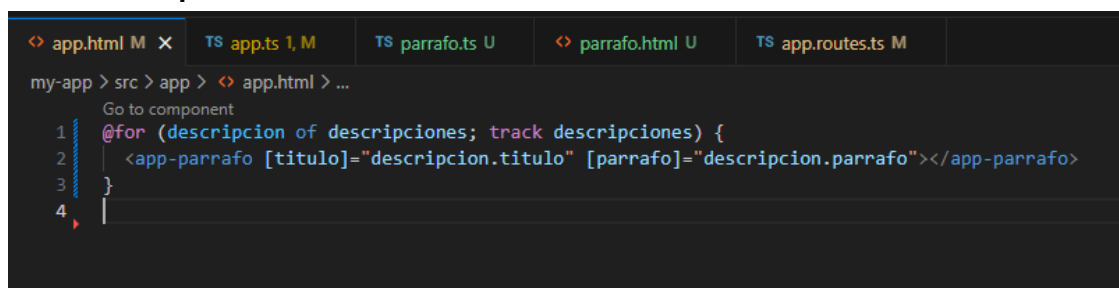


```
my-app > src > app > TS app.ts > App > descripciones
1  import { Component } from '@angular/core';
2  import { RouterOutlet } from '@angular/router';
3  import { Parrafo } from "../parrafo/parrafo";
4
5  @Component({
6    selector: 'app-root',
7    imports: [RouterOutlet, Parrafo], RouterOutlet is not used within the component
8    templateUrl: './app.html',
9    styleUrls: ['./app.css']
10 })
11 export class App {
12   protected title = 'my-app';
13
14   descripciones = [
15     {titulo: "Titulo 1", parrafo: "Este es el texto numero 1"},
16     {titulo: "Titulo 2", parrafo: "Este es el texto numero 2"},
17     {titulo: "Titulo 3", parrafo: "Este es el texto numero 3"},
18   ];
19
20 }
21
```

En **app.html** vamos a reflejar este cambio, y de paso agregarle los parámetros al componente.

Aca tenemos cosas nuevas, recordemos que **descripciones** viene de [app.ts](#) (Antes era *cant*), por lo que en el **for** vamos a estar **iterando objetos** que tienen la estructura título y párrafo, son los **i.titulo** e **i.parrafo**.

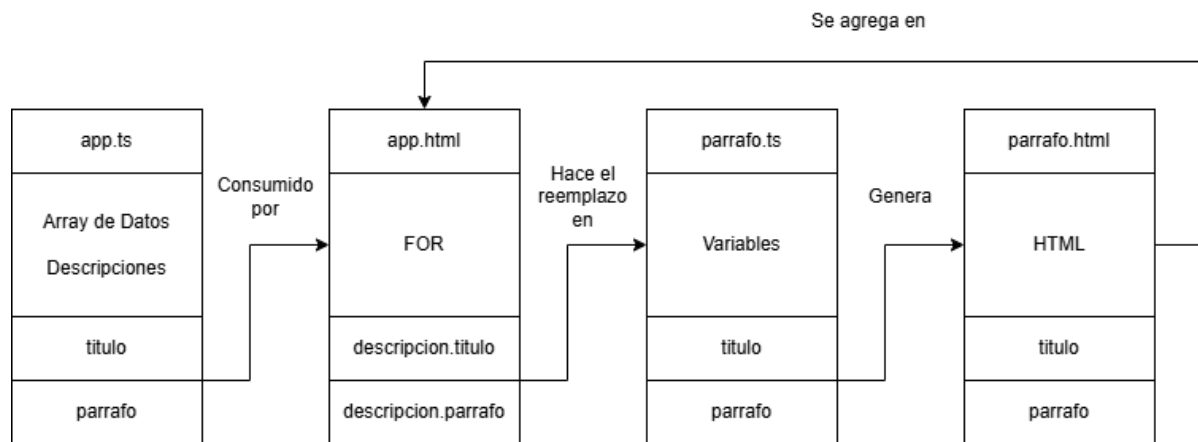
Lo que esta entre [] es la variable que vamos a reemplazar en **app-parrafo**, la que definimos en **parrafo.ts**.



```
my-app > src > app > app.html > ...
Go to component
1  @for (descripcion of descripciones; track descripciones) {
2    <app-parrafo [titulo]="descripcion.titulo" [parrafo]="descripcion.parrafo"></app-parrafo>
3  }
4
```

La conexión entre datos sería algo así:

1. Primero [app.ts](#) **generan los datos** que vamos a usar, que en nuestro caso es una array de objetos con título y párrafo.
2. Esto es **consumido por app.html** que en el bucle for itera sobre cada uno de los elementos que debemos en ese array, mandando el contenido de sus componentes **para que reemplace las variables que definimos en [parrafo.ts](#)**.
3. Con la **variable ya reemplazada genera una copia de lo que tenemos definido en [parrafo.html](#)** pero reemplazando el contenido de los elementos entre `{{ }}` con su correspondiente variable previamente reemplazada
4. Esto se **agrega en la vista principal** como varios elementos con sus partes modificadas basadas en la plantilla definida en [parrafo.html](#)



Finalmente queda así

Titulo 1

Este es el texto numero 1

Titulo 2

Este es el texto numero 2

Titulo 3

Este es el texto numero 3

Eventos

Supongamos que queremos que al hacer click sobre el título nos envíe una alerta diciendo que tituló clickeamos y cual es su contenido. Pero queremos que también sea genérico para cada elemento generado. Esto lo podemos hacer emitiendo eventos desde un elemento hijo (**Todo aquel que no sea [app.ts](#)**)

Primero vamos a **definir el evento al hacer click** sobre un **elemento** en **parrafo.html** ya que es sobre estos elementos donde queremos accionar el evento.

```
parrafo.html M X  app.html M  TS app.ts 1, M  TS parrafo

my-app > src > app > parrafo > < parrafo.html > ...
Go to component
1  <!--
2  <h1>{{titulo()}}</h1>
3  <p>{{parrafo()}}</p>
4  -->
5
6  <h1 (click)="eventoAlerta()">{{titulo}}</h1>
7  <p>{{parrafo}}</p>
8  |
```

Luego vamos a tener que **preparar los datos** que queremos cargar en ese evento, esto lo hacemos en [parrafo.ts](#). Acá lo que queremos es decirle al modulo que esta vez **no reciba datos, sino que los emita**, mediante el uso de **@Output()** seguido del **nombre que le queremos dar al evento**, para luego **instanciarlo con un tipo de dato que queramos**:

```
@Output() nombre_evento= new EventEmitter<tipo_dato>();
```

Lo último que hay que hacer es decirle a este evento que datos tiene que tener definiendo la función utilizada en **parrafo.html**

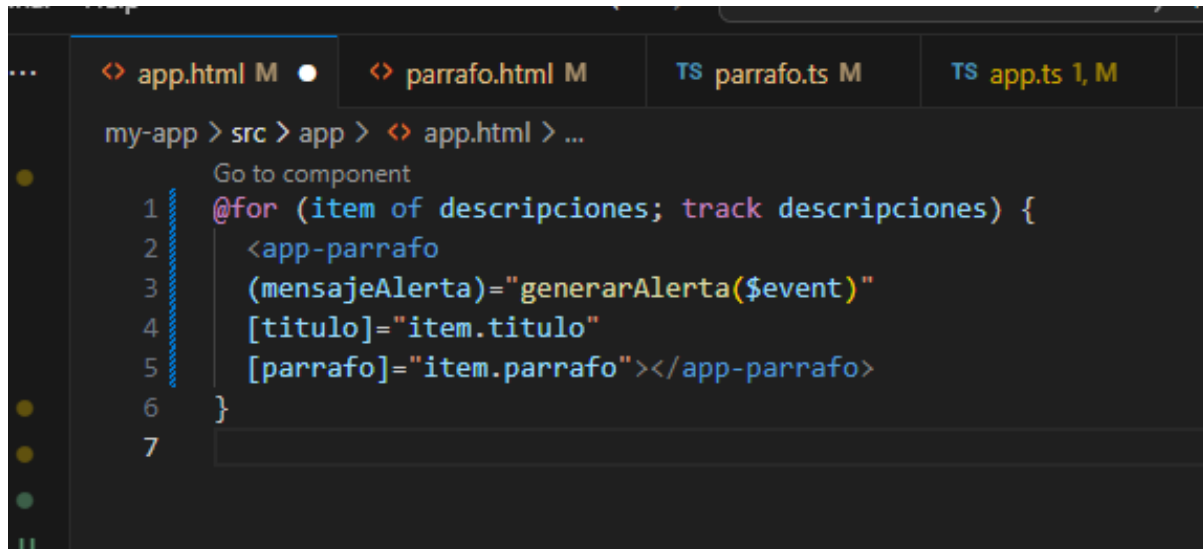
```
parrafo.html M  TS parrafo.ts M X  app.html M  TS app.ts 1, M

my-app > src > app > parrafo > TS parrafo.ts > Parrafo > mensajeAlerta
1  import { Component, Input, input, Output, EventEmitter } from '@angular/
2  import { Descripcion } from '../models/descripcion.interface';
3
4
5  @Component({
6    selector: 'app-parrafo',
7    imports: [],
8    templateUrl: './parrafo.html',
9    styleUrls: ['./parrafo.css']
10 })
11 export class Parrafo {
12   //titulo = input.required<string>();
13   //parrafo = input.required<string>();
14
15   @Input() titulo: string = '';
16   @Input() parrafo: string = '';
17
18   @Output() mensajeAlerta = new EventEmitter<Descripcion>();
19
20   eventoAlerta() {
21     this.mensajeAlerta.emit({
22       titulo: this.titulo,
23       parrafo: this.parrafo
24     });
25   }
26 }
27
```

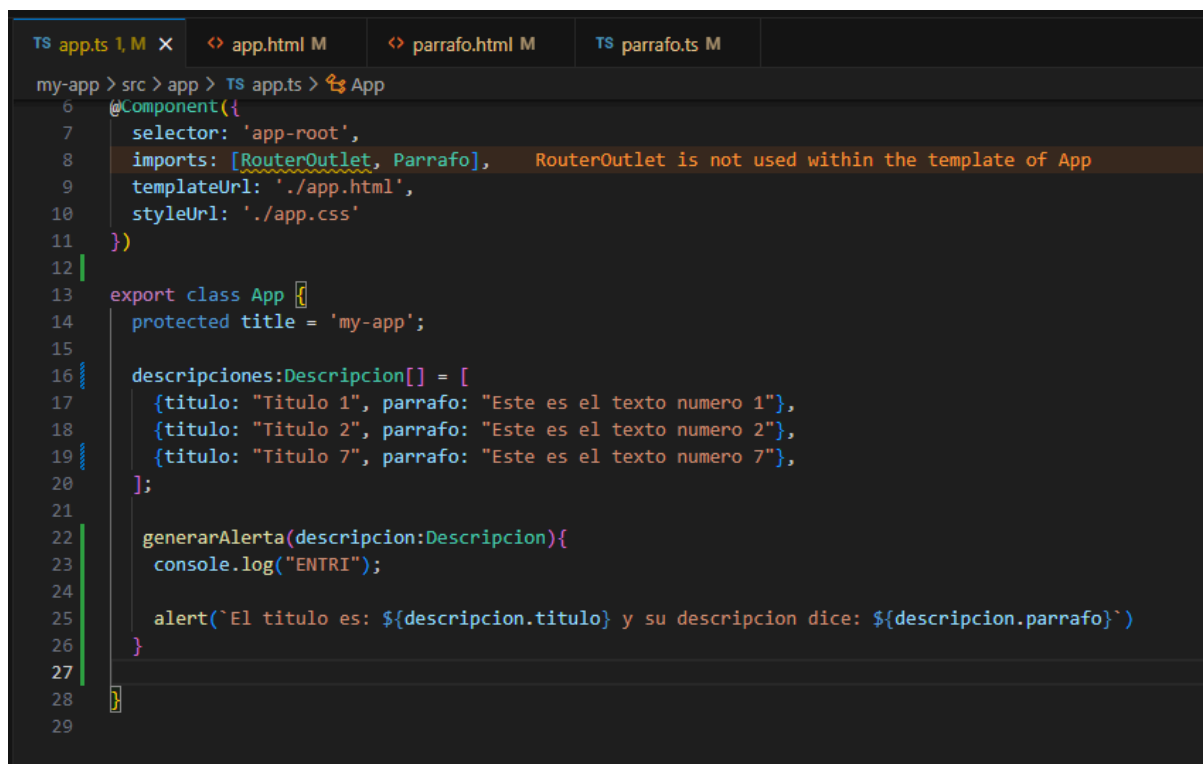
Ahora vamos a **modificar la app.html** para que **capture ese evento** y **ejecute una función en consecuencia**. Esto es similar a el evento del click que usamos en parrafo.html, solo que esta vez el **evento va a tener el nombre que le definimos nosotros**, y la **función que ejecuta la definiremos en app.ts**.

Los datos que cargamos en [parrafo.ts](#) son capturables mediante \$event

Nótese en la foto que **mensajeAlerta** es el nombre que definimos en @Outpu() en parrafo.ts



```
my-app > src > app > <> app.html > ...
Go to component
1  @for (item of descripciones; track descripciones) {
2    <app-parrafo
3      (mensajeAlerta)="generarAlerta($event)"
4      [titulo]="item.titulo"
5      [parrafo]="item.parrafo"></app-parrafo>
6  }
7
```



```
TS app.ts 1, M X  <> app.html M  <> parrafo.html M  TS parrafo.ts M
my-app > src > app > TS app.ts > App
6  @Component({
7    selector: 'app-root',
8    imports: [RouterOutlet, Parrafo], RouterOutlet is not used within the template of App
9    templateUrl: './app.html',
10   styleUrls: ['./app.css']
11 })
12
13 export class App {
14   protected title = 'my-app';
15
16   descripciones: Descripcion[] = [
17     {titulo: "Titulo 1", parrafo: "Este es el texto numero 1"},
18     {titulo: "Titulo 2", parrafo: "Este es el texto numero 2"},
19     {titulo: "Titulo 7", parrafo: "Este es el texto numero 7"},
20   ];
21
22   generarAlerta(descripcion: Descripcion){
23     console.log("ENTRI");
24
25     alert(`El titulo es: ${descripcion.titulo} y su descripcion dice: ${descripcion.parrafo}`)
26   }
27
28 }
29
```

La conexión entre datos seria algo asi:

1. En **parrafo.html** definimos que **escuche el evento click** sobre un elemento.
2. Esto **ejecuta una función** definida en [parrafo.ts](#), dicha función **prepara los datos para emitir un evento personalizado**.
3. Los elementos ya generados en **app.html** **escuchan este evento**, **capturan los datos y ejecutan una función**.
4. Utilizando los datos del evento que le llegan por parámetro realiza operaciones

